Composite Quadrature Methods

Jonathan Cushman

April 2026

1 Experimental Design

This project investigates the numerical integration of five independent test functions using three composite quadrature methods: The composite trapezoid rule, an adaptive composite trapezoidal rule with step halving refinements, and the composite two-point Gauss-Legendre rule.

The metrics used to evaluate the methods in this experiment include

- **True Error**: $E_{true} = |I_{true} - I_{computed}|$
- **Observed Error**: $E_{observed} = |I_{fine} - I_{coarse}|$
- **A Priori Bound**: $|E| \leq \frac{b-a}{12} H^2 \max |f''(x)| \leq \epsilon$

The true error is the actual error between the numerical solution and the known analytical solution. The observed error is a practical error estimate between numerical solutions. This is further used as the stopping criteria in the adaptive method by comparing runs with different step sizes. The a priori bound $= \epsilon$ is the error that can be guaranteed before running any experiments. H can be specifically derived to produce an expected error at most ϵ.

The following Integrals and their known solutions were used to empirically validate the numerical solutions.

- **Test function 1**: $\int_0^3 e^x \, dx = e^3 - 1$
- **Test function 2**: $\int_0^{\frac{\pi}{3}} e^{\sin(2x)} \cos(2x) \, dx = \frac{1}{2}(-1 + e^{\frac{\sqrt{3}}{2}})$
- **Test function 3**: $\int_{-2}^1 \tanh(x) \, dx = ln(\frac{\cosh(1)}{\cosh(2)})$
- **Test function 4**: $\int_0^{3.5} x \cos(2\pi x) \, dx = -\frac{1}{2\pi^2}$
- **Test function 5**: $\int_{0.1}^{2.5} (x + \frac{1}{x}) \, dx = \frac{2.5^2 - 0.1^2}{2} = ln(\frac{2.5}{0.1})$

1.1 Sub-Interval Size Estimation

The main objective of this task is to predict the required sub-interval width H_m using the known error bound for the composite trapezoid method. This is done by rearranging the a priori bound and solving for H_m such that

$$H_m \leq \sqrt{\frac{12\epsilon}{(b-a)\max|f''(x)|}} \quad where \quad m = \frac{b-a}{H_m} \tag{1}$$

The maximum of the second derivative is estimated numerically using the built in finite difference method from the Numpy package, evaluated at 1000 uniformly spaced point across the intervals. Although these values are not perfect approximations, I found it useful to play around with this function for potential future use.

The results for testing are shown below for $\epsilon = [10^{-2}, 10^{-4}, 10^{-6}, 10^{-8}]$

Table 1: A Priori Sub-interval Estimations

Function	**max**$\lvert f''\rvert$	**Tol.**	H_m	m
e^x on $[0,3]$	20.0253	10^{-2}	0.044693	68
		10^{-4}	0.004469	672
		10^{-6}	0.000447	6713
		10^{-8}	0.000045	67125
$e^{\sin(2x)}\cos(2x)$ on $[0,\pi/3]$	16.2830	10^{-2}	0.083890	13
		10^{-4}	0.008389	125
		10^{-6}	0.000839	1249
		10^{-8}	0.000084	12484
$\tanh(x)$ on $[-2,1]$	0.7698	10^{-2}	0.227951	14
		10^{-4}	0.022795	132
		10^{-6}	0.002280	1317
		10^{-8}	0.000228	13161
$x\cos(2\pi x)$ on $[0,3.5]$	137.7206	10^{-2}	0.015778	222
		10^{-4}	0.001578	2219
		10^{-6}	0.000158	22183
		10^{-8}	0.000016	221826
$x + \frac{1}{x}$ on $[0.1, 2.5]$	1863.5397	10^{-2}	0.005180	464
		10^{-4}	0.000518	4634
		10^{-6}	0.000052	46334
		10^{-8}	0.000005	463336

The results from testing show a clear relationship between the magnitude of the second derivative value and the number of sub-interval needed to match the tolerance value. As the magnitude of the second derivative rises, the number and size of the sub-intervals scales along side this value. This follows directly from the quadratic dependence on H in the error bound.

This is also associated with m the number of sub-intervals needed to reach the target tolerance. Each time the tolerance ϵ is decreased by a factor of 100, the number of sub-intervals used increases by a factor of 10 for each function. This is directly related to the H^2 term in the a priori bound, which claims that in order to reduce the bound by 100 times, the size of H needs $\sqrt{100} = 10$ times as many sub-intervals. This factor is consistent with the testing for each method and confirms the second order convergence rate of the trapezoidal rule is being captured.

1.2 Composite Trapezoidal Rule Testing

Using the predicted number of sub-intervals m, the next objective is computing the true and observed error for the composite trapezoidal method using the formulas from the experimental design section. The results for testing are shown below.

Table 2: Composite Trapezoidal Rule Testing

Function	**Tolerance**	m	**True Error**	**Observed Error**
e^x	10^{-2}	68	3.0955e-03	2.3216e-03
	10^{-4}	672	3.1698e-05	2.3773e-05
	10^{-6}	6713	3.1764e-07	2.3823e-07
	10^{-8}	67125	3.1767e-09	2.3823e-09
$e^{\sin(2x)}\cos(2x)$	10^{-2}	13	2.6678e-03	2.0013e-03
	10^{-4}	125	2.8829e-05	2.1622e-05
	10^{-6}	1249	2.8875e-07	2.1656e-07
	10^{-8}	12484	2.8903e-09	2.1677e-09
$\tanh(x)$	10^{-2}	14	1.3356e-03	1.0015e-03
	10^{-4}	132	1.5036e-05	1.1277e-05
	10^{-6}	1317	1.5105e-07	1.1329e-07
	10^{-8}	13161	1.5126e-09	1.1344e-09
$x\cos(2\pi x)$	10^{-2}	222	4.1447e-05	3.1089e-05
	10^{-4}	2219	4.1464e-07	3.1098e-07
	10^{-6}	22183	4.1490e-09	3.1118e-09
	10^{-8}	221826	4.1492e-11	3.1123e-11
$x + \frac{1}{x}$	10^{-2}	464	2.2253e-04	1.6689e-04
	10^{-4}	4634	2.2317e-06	1.6738e-06
	10^{-6}	46334	2.2323e-08	1.6742e-08
	10^{-8}	463336	2.2340e-10	1.6761e-10

As seen by the results, the true and observed error are always strictly lower than the a priori bound (tolerance) for each of the functions tested. This relationship from the results shows that

$$ObservedError < TrueError < APrioriBound(Tol.) \tag{2}$$

The observed error is slightly smaller than the true error due to both the fine and course approximations being on the same side of the exact answers. The difference between the two underestimates the actual gap. The true error on the other hand is much smaller than the a priori bound due to the fact that the bound uses the worst case possible given by the max $f''(x)$ over the interval, causing the bound to be more cynical and downbeat which leaves some breathing room in the bound.

1.3 Adaptive Composite Trapezoidal Rule Testing

The objective for the next task follows as from before but uses the adaptive composite trapezoidal rule with step-halving intervals. Unlike before, the adaptive method finds it's own values of m to hit to desired tolerance from the a priori bound. The results are as follows below

Table 3: Adaptive Composite Trapezoidal Rule Testing

Function	**Tol.**	m	**True Error**	**Observed Error**	**Evals**
e^x	10^{-2}	128	8.7366e-04	1.3103e-02	263
	10^{-4}	1024	1.3651e-05	2.0477e-04	2058
	10^{-6}	8192	2.1330e-07	6.3989e-07	16397
	10^{-8}	65536	3.3329e-09	9.9981e-09	131088
$e^{\sin(2x)}\cos(2x)$	10^{-2}	16	1.7606e-03	2.6652e-02	36
	10^{-4}	128	2.7494e-05	8.2484e-05	263
	10^{-6}	2048	1.0740e-07	1.6109e-06	4107
	10^{-8}	16384	1.6781e-09	2.5171e-08	32782
$\tanh(x)$	10^{-2}	16	1.0228e-03	1.5175e-02	36
	10^{-4}	128	1.5991e-05	2.3982e-04	263
	10^{-6}	1024	2.4986e-07	7.4957e-07	2058
	10^{-8}	16384	9.7600e-10	1.4640e-08	32782
$x\cos(2\pi x)$	10^{-2}	32	2.0418e-03	6.7472e-03	69
	10^{-4}	256	3.1165e-05	9.3633e-05	520
	10^{-6}	4096	1.2169e-07	1.8254e-06	8204
	10^{-8}	32768	1.9014e-09	2.8522e-08	65551
$x + \frac{1}{x}$	10^{-2}	128	2.9149e-03	8.6301e-03	263
	10^{-4}	2048	1.1426e-05	1.7135e-04	4107
	10^{-6}	16384	1.7853e-07	2.6779e-06	32782
	10^{-8}	131072	2.7894e-09	8.3685e-09	262161

Unlike before, the relationship between the true error and the observed error does not hold from the test results. Additionally comparing the number of sub-intervals m, test functions 2,3 used slightly more sub-intervals to achieve a comparable error between the methods. The other tested functions used less intervals as compared to the previous method, with a substantial drop off between functions 4 and 5. Thus

showcasing the superiority of using the adaptive method for cases when the sub-interval blows up in the standard composite method.

The effect of halving the interval naturally discovers that the integral converges faster than the bound suggests and stops earlier than needed. This demonstrates the tradeoff between the a priori analysis and the adaptive refinement. The analysis is more effective when f"(x) accurately represents the global difficulty, while the local refinement is better for bounds that are locally dominated by narrow regions of high curvature.

1.4 Composite Two-Point Gauss-Legendre Testing

The last section of testing includes the results for the two-point Gauss-Legendre method. This test is run for increasing values of m until the true error is below the tolerance threshold. The number of function evaluations are then tracked and compared against the previous methods. Below are the results of testing

Table 4: Composite Gauss-Legendre Testing vs Trapezoidal Methods

Func.	**Tol.**	m	**GL Evals**	**True Error**	**Trap Evals**	**Adapt Evals**
1	10^{-2}	3	6	4.2813e-03	69	263
	10^{-4}	8	16	8.6978e-05	673	2058
	10^{-6}	25	50	9.1569e-07	6714	16397
	10^{-8}	78	156	9.6673e-09	67126	131088
2	10^{-2}	2	4	4.4001e-04	14	36
	10^{-4}	4	8	4.2786e-05	126	263
	10^{-6}	11	22	7.7175e-07	1250	4107
	10^{-8}	33	66	9.5566e-09	12485	32782
3	10^{-2}	2	4	4.4282e-03	15	36
	10^{-4}	4	8	2.9872e-05	133	263
	10^{-6}	10	20	7.2483e-07	1318	2058
	10^{-8}	29	58	9.8367e-09	13162	32782
4	10^{-2}	7	14	6.1793e-03	223	69
	10^{-4}	18	36	8.4932e-05	2220	520
	10^{-6}	54	108	9.7621e-07	22184	8204
	10^{-8}	170	340	9.8603e-09	221827	65551
5	10^{-2}	12	24	7.9229e-03	465	263
	10^{-4}	45	90	9.6509e-05	4635	4107
	10^{-6}	146	292	9.9725e-07	46335	32782
	10^{-8}	464	928	9.9244e-09	463337	262161

The test results indicate that the Gauss-Legendre method outperforms both trapezoidal methods without question. The number of sub-intervals and evaluations needed to hit the desired tolerance level are magnitudes of order lower than the previous

methods. The Gauss-Legendre method is able to capture the functions behavior more accurately with fewer point per sub-interval. This helps to showcase the fourth-order convergence associated with the method where it significantly outperforms the aforementioned methods above.

1.5 Convergence Plots

To further illustrate the results, plots for each technique on the individual test function are included. These plots help to showcase the convergence rate in real time for each method. Each plot compares the true error against the number of function evaluations. Both of the axis's are scaled logarithmically as the data spans several orders of magnitude.

Figure 1: Convergence for Test Function 1

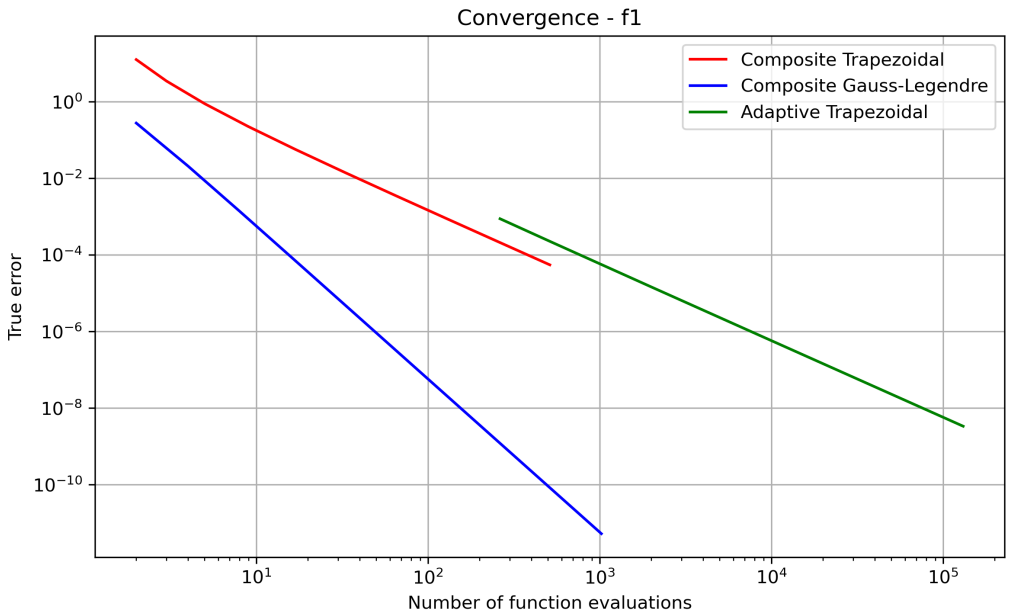

Figure 2: Convergence for Test Function 2

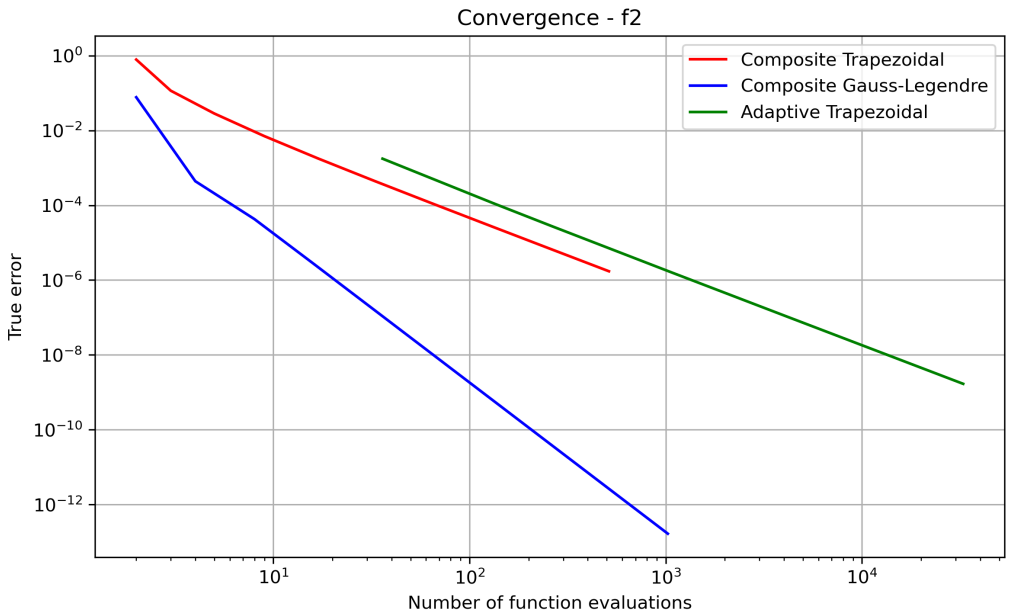

Figure 3: Convergence for Test Function 3

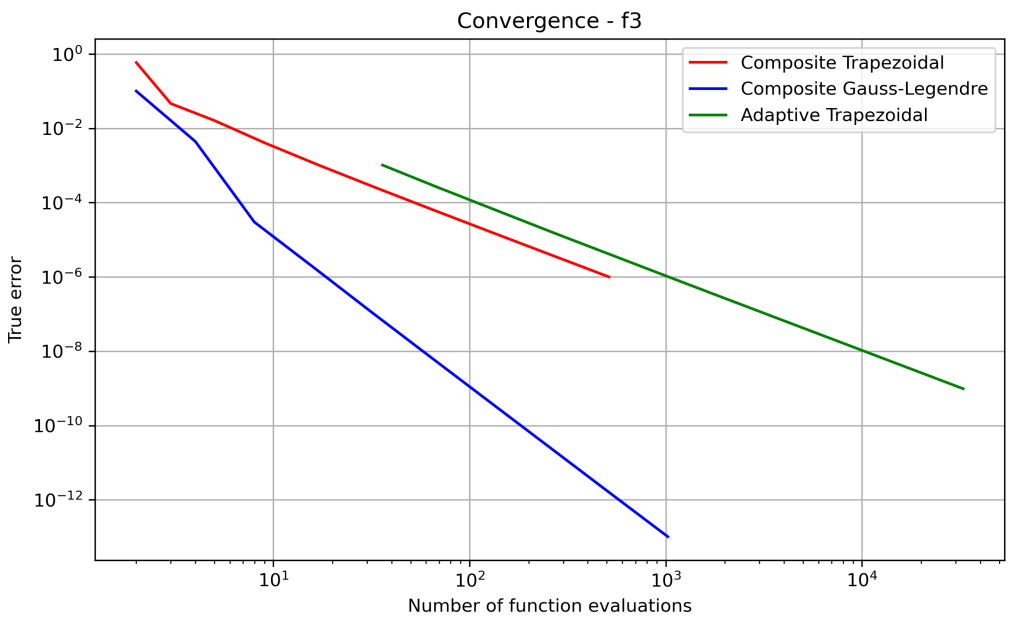

Figure 4: Convergence for Test Function 4

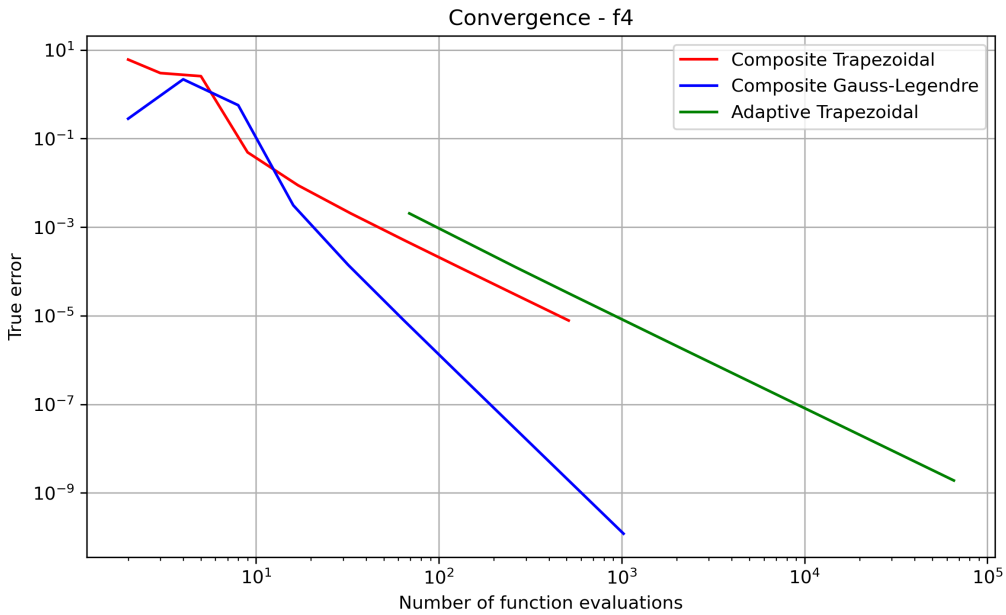
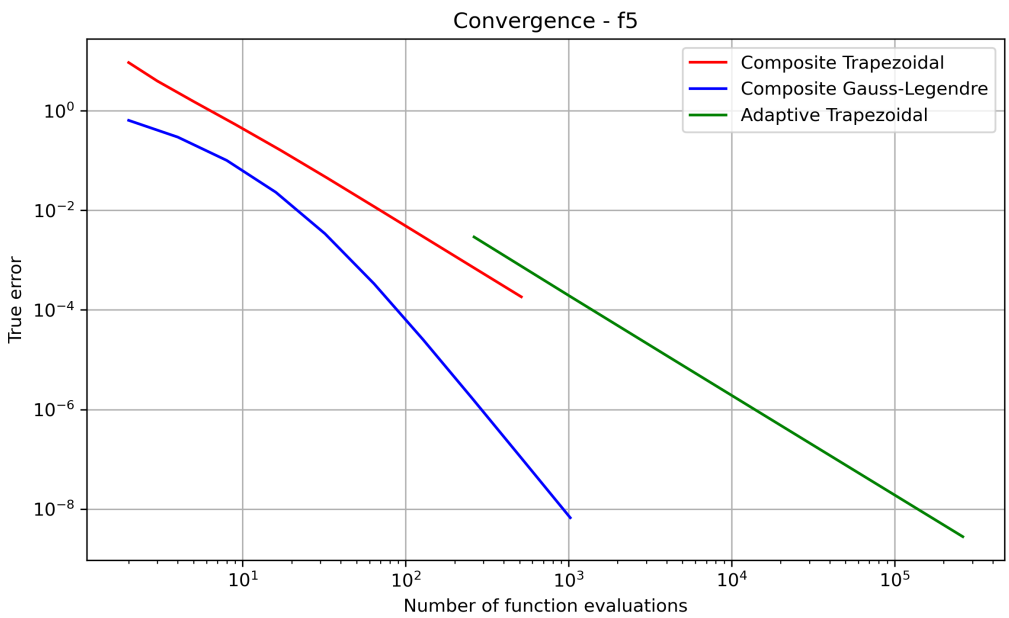
Figure 5: Convergence for Test Function 5